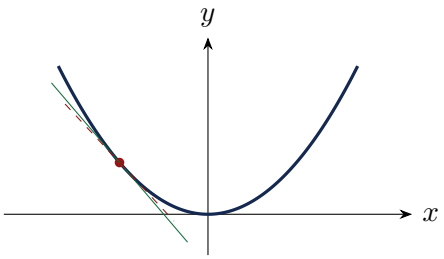
Foundations of Machine Learning

A Book for Absolute Beginners

From basic maths → machine learning, derived step by step

No prior maths assumed — everything built from zero



Prepared by

Krushna Parmar

Roll No. **202611032** • Course IT582

Grounded in the course lectures and class notes. Every idea is explained in plain words first, then derived step by step, then drawn as a picture — with analogies and practice questions. Built for someone starting from nothing.

Contents

How to Read This Book

This book assumes **you know almost no maths**, and builds everything up. Each idea appears in four forms so at least one clicks:

- **In Plain Words** (green) — the idea with zero jargon, first.
- **Derivation — step by step** (blue) — every algebra line, nothing skipped.
- **Figure** — a picture in the spirit of 3Blue1Brown, so you can *see* the maths.
- **Analogy** (green) and **Chai-Break** (orange) — everyday comparisons to make it stick.
- **Questions & Variations** (purple) — what an examiner might ask, and the twists.
- **Watch Out** (red) — the mistakes beginners make.

Read Chapter 0 first even if it looks basic — every later chapter uses it. If a symbol ever confuses you, it is defined in Chapter 0.

Chapter 1

The Math You Need, From Zero

Before any machine learning, we need a handful of maths ideas. We build each from nothing, with a picture. If you already know one, skim it.

1.1 Numbers, variables and functions

In Plain Words

A **variable** is just a name for a number we do not want to fix yet — like a labelled empty box. If I say "let x be your age", then x stands for whatever number that is. A **function** is a machine: put a number in, get a number out, by a fixed rule.

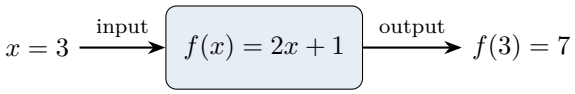


Figure — A function is a machine: it takes an input number, applies a rule, and returns an output number.

Concept

We write a function as $f(x) =$ (some rule using x). The symbol $f(3)$ means "run the machine on input 3". For $f(x) = 2x + 1$: $f(3) = 2 \cdot 3 + 1 = 7$. The letter f is just a name; we could call it g, h, or — in machine learning — the *hypothesis* h.

1.2 Drawing a function: the graph

In Plain Words

To *see* a function, we draw it. We make a flat sheet with a horizontal line (the x-axis, inputs) and a vertical line (the y-axis, outputs). For each input x we go right by x and up by $f(x)$, and mark a dot. Join the dots — that curve *is* the function.

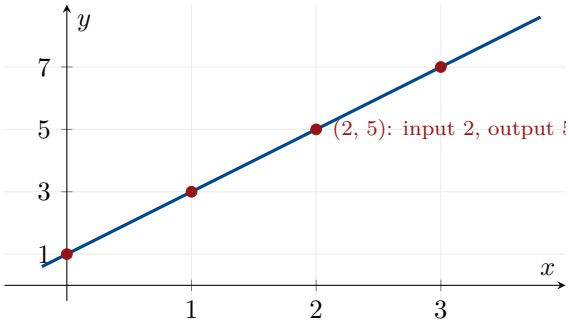


Figure — The graph of $f(x) = 2x + 1$. Each red dot is one (input, output) pair; the line is the whole function at a glance.

1.3 Slope: how steep is a line?

In Plain Words

The **slope** of a straight line is how much it rises when you walk one step to the right. Big slope = steep climb; zero slope = flat; negative slope = going downhill. It is "rise over run": how far up divided by how far across.

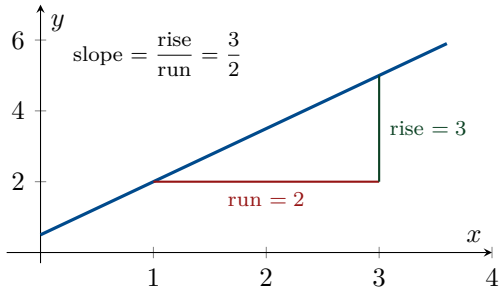


Figure — Slope = rise ÷ run. Walk 2 across, climb 3 up ⇒ slope = 1.5. This is the single most important idea in this book.

Concept

For a line through two points (x_1, y_1) and (x_2, y_2),

$$\text{slope} = \frac{y_2 - y_1}{x_2 - x_1} = \frac{\text{change in } y}{\text{change in } x}.$$

A line's equation is $y = mx + c$, where m is the slope and c is where it crosses the y-axis (the "intercept"). In machine learning this exact line becomes our first model.

1.4 The derivative: slope of a curve

In Plain Words

A curve is not equally steep everywhere. The **derivative** is the slope *at a single point* — the steepness you would feel if you stood right there. We find it by zooming in: pick the point, pick a second point very close by, take rise-over-run, then slide the second point in until they almost touch.

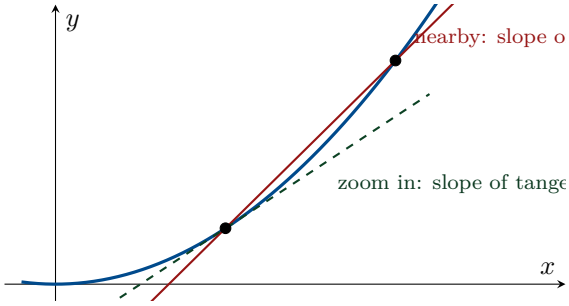


Figure — Slide the far point toward the near one: the red "secant" rotates into the green "tangent". That tangent's slope is the derivative at the point.

Derivation — step by step

Derivative of $f(x) = x^2$, from scratch. Take a point x and a tiny step h:

$$\begin{aligned}\text{slope near } x &= \frac{f(x+h) - f(x)}{h} = \frac{(x+h)^2 - x^2}{h} \\ &= \frac{x^2 + 2xh + h^2 - x^2}{h} && \text{(expand the square)} \\ &= \frac{2xh + h^2}{h} = 2x + h. && \text{(cancel } x^2\text{, divide by } h\text{)}\end{aligned}$$

Now let the step h shrink to 0: the leftover h vanishes, leaving the slope $2x$. So the derivative of x^2 is $\boxed{2x}$. (We write it $f'(x) = 2x$ or $\frac{df}{dx} = 2x$.)

Concept

The power rule generalises this: $\dfrac{d}{dx}x^n = n\,x^{n-1}$. A constant multiplier rides along ($\frac{d}{dx}(c\,x^n) = c\,n\,x^{n-1}$), you differentiate a sum term by term, and a lone constant has slope 0 (a flat line). These few rules cover everything in this course.

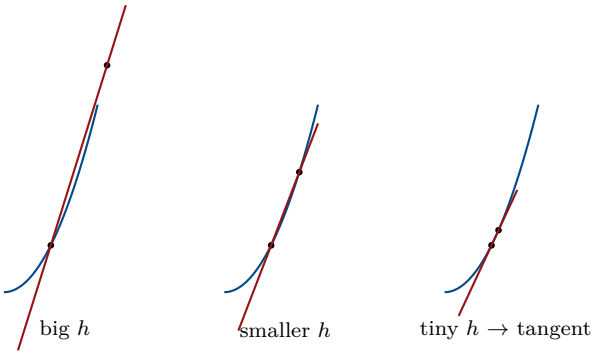


Figure — Zooming in: as the step h shrinks, the red secant line through the two dots rotates until it becomes the tangent — its slope is the derivative. This is what "$\lim_{h\to 0}$" means, drawn.

Analogy

The derivative is your car's **speedometer**. The road-position over time is the function; the speedometer shows the slope *right now* — how fast position is changing at this instant. Speeding up, slowing down, or cruising flat are positive, negative, and zero derivatives.

1.5 Vectors: numbers with direction

In Plain Words

A **vector** is just a list of numbers, e.g. $(3, 2)$. You can picture it as an *arrow*: start at the origin, go 3 right and 2 up. In machine learning, one data point (say a house's features) is a vector — a list of its measurements.

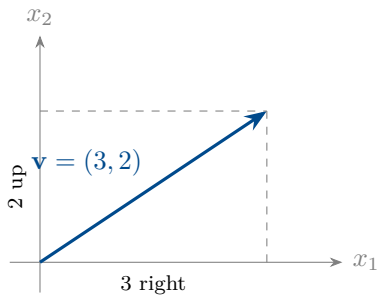


Figure — A vector as an arrow: $(3, 2)$ means "3 along, 2 up". A list of numbers and an arrow are the same thing.

Figure — Add vectors by placing them tip-to-tail; scale a vector by stretching it (the direction stays, the length changes).

Concept

Dot product. For $\mathbf{a} = (a_1, a_2)$ and $\mathbf{b} = (b_1, b_2)$, the dot product is $\mathbf{a} \cdot \mathbf{b} = a_1 b_1 + a_2 b_2$ — multiply matching entries and add. It gives a single number that measures how much the two arrows point the same way. In ML, a prediction is often a dot product of a weight vector with a feature vector.

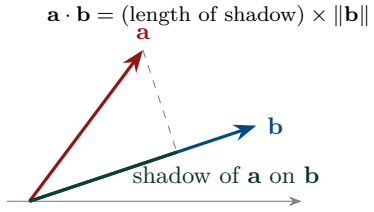


Figure — The dot product, geometrically: drop $\mathbf{a}$'s shadow onto $\mathbf{b}$. Aligned arrows ⇒ big positive; perpendicular ⇒ 0; opposite ⇒ negative.

Derivation — step by step

Worked numbers. For $\mathbf{a} = (1, 2)$, $\mathbf{b} = (3, 4)$: $\mathbf{a} \cdot \mathbf{b} = 1 \cdot 3 + 2 \cdot 4 = 11$. A **length** is the

dot product of a vector with itself, square-rooted: for $x = (2, 3, 4)$,

$$\|x\| = \sqrt{x \cdot x} = \sqrt{2^2 + 3^2 + 4^2} = \sqrt{29} \approx 5.39.$$

(This exact $\sqrt{29}$ appears in the class notes — it measures "how far did θ move?" in the stopping rules of Chapter 4.)

How to write it in Python

Every vector operation is one line of NumPy:

```
import numpy as np
a = np.array([1, 2]);  b = np.array([3, 4])
print(a + b)                  # [4 6]   addition (tip-to-tail)
print(2 * a)                  # [2 4]   scaling
print(a @ b)                  # 11      dot product = 1*3 + 2*4
x = np.array([2, 3, 4])
print(np.linalg.norm(x))      # 5.385... = sqrt(29), the length of x
```

The `@` operator *is* the dot product; `np.linalg.norm` gives $\|x\|$.

1.6 Matrices: tables that transform vectors

In Plain Words

A **matrix** is a grid of numbers (rows and columns). One use: it stores a whole dataset — one row per example, one column per feature. Another use: multiplying a matrix by a vector transforms that vector (rotates/stretches the arrow). For us, matrices let us write "do this to every data point at once" in one short symbol.

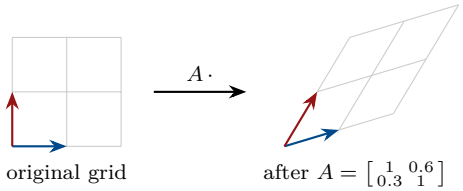


Figure — A matrix transforms space: it takes the square grid (and the basis arrows) and stretches/shears it into a new grid. Multiplying Ax moves the point x to its new spot.

$$\underbrace{\begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ 1 & x_3 \end{bmatrix}}_{X\ (3\times 2)} \underbrace{\begin{bmatrix} \theta_0 \\ \theta_1 \end{bmatrix}}_{\theta} = \begin{bmatrix} \theta_0 + \theta_1 x_1 \\ \theta_0 + \theta_1 x_2 \\ \theta_0 + \theta_1 x_3 \end{bmatrix}$$

Figure — Matrix × vector: each row of X dotted with θ gives one prediction. One symbol, all examples at once — this is exactly how we will predict house prices.

Concept

To multiply a matrix by a vector, take the **dot product of each row** with the vector. An $(m \times n)$ matrix times an $(n \times 1)$ vector gives an $(m \times 1)$ vector — the inner sizes (n) must match. The transpose $X^\top$ flips rows into columns; it appears whenever we "fold

errors back onto parameters" (Chapter 5).

Derivation — step by step

Worked matrix–vector multiply. Take the design matrix and a parameter vector:

$$X = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix}, \quad \theta = \begin{bmatrix} 0.5 \\ 2 \end{bmatrix} \Rightarrow X\theta = \begin{bmatrix} 1(0.5) + 1(2) \\ 1(0.5) + 2(2) \\ 1(0.5) + 3(2) \end{bmatrix} = \begin{bmatrix} 2.5 \\ 4.5 \\ 6.5 \end{bmatrix}.$$

Each row of X is dotted with θ — so one matrix multiply produces *all three* predictions at once. That is the whole reason we use matrices.

How to write it in Python

The same multiply, in NumPy:

```
import numpy as np
X = np.array([[1, 1],
              [1, 2],
              [1, 3]])          # rows = examples; first column of 1s
                 = bias
theta = np.array([0.5, 2.0])    # intercept 0.5, slope 2.0
print(X @ theta)                # [2.5 4.5 6.5]  one prediction per
    row
```

`X @ theta` dots each row with `theta` — exactly $X\theta$.

1.7 Summation notation $\sum$

In Plain Words

The big Greek Σ ("sigma") just means **add up a list**. It is shorthand so we do not write "+" a hundred times.

$$\sum_{i=1}^{4} a_i \xrightarrow{\text{unroll}} = a_1 + a_2 + a_3 + a_4$$

Figure — $\sum_{i=1}^{4} a_i$ reads "add a_i for i from 1 to 4". The bottom is where to start, the top where to stop.

Concept

The **mean** (average) of m numbers uses it: $\bar{a} = \frac{1}{m}\sum_{i=1}^{m} a_i$ — add them all, divide by how many. Our main error measure, the Mean Squared Error, is just the mean of squared mistakes: $\frac{1}{m}\sum_{i=1}^{m}(\text{mistake}_i)^2$.

Questions & Variations

Q. What does $f(2)$ mean for $f(x) = 3x - 1$?
A. Run the machine on input 2: $3 \cdot 2 - 1 = 5$.

Q. What is the slope of the line through $(0, 1)$ **and** $(2, 5)$**?**
A. $\frac{5-1}{2-0} = 2$.

Q. What is the derivative of x^3**?**
A. By the power rule, $3x^2$.

Q. Compute the dot product $(1, 2) \cdot (3, 4)$**.**
A. $1 \cdot 3 + 2 \cdot 4 = 3 + 8 = 11$.

Q. Why do we use matrices in ML?
A. To apply the same computation to every data point at once, written compactly (e.g. all predictions as $X\theta$).

Chapter 2

What is Machine Learning?

Now that the maths tools are in hand, we can say precisely what machine learning *is* — using the exact definitions and the running example from the course lectures.

2.1 Two ways to make a computer do a task

In Plain Words

Normally, to make a computer do something, *you* write the rules: "if the email contains 'lottery', mark it spam". That is **traditional programming** — you supply the rules, the computer applies them. **Machine learning flips this around**: you show the computer lots of *examples* (emails already labelled spam / not-spam) and it *figures out the rules itself*.

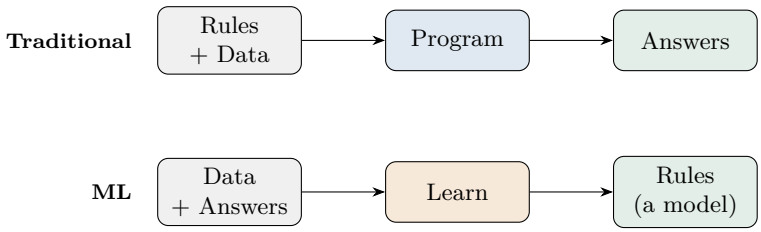


Figure — Traditional programming turns rules + data into answers. Machine learning turns data + answers into the rules (a "model").

2.2 Two classic definitions

Concept

The course gives two famous definitions:

- **Arthur Samuel (1959):** "The field of study that gives computers the ability to learn *without being explicitly programmed*."
- **Tom Mitchell (1998):** "A computer program is said to learn from experience **E** with respect to some class of tasks **T** and performance measure **P**, if its performance at tasks in T, as measured by P, *improves with experience* E."

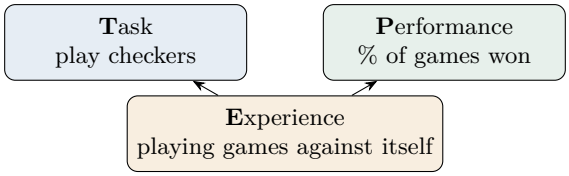


Figure — Mitchell's T/P/E, for a checkers program: more games played (E) improves the win-rate (P) on the task (T). That improvement is learning.

Analogy

This is exactly how **a child learns to ride a bicycle**. Nobody writes the child a rulebook of muscle movements (that would be traditional programming). The child *tries* (experience E), *wobbles and falls* (performance P), and *improves* with practice (task T). Machine learning is a computer doing the same: getting better at a task through experience.

2.3 The big idea: learning = fitting a function

In Plain Words

Underneath, a machine-learning model is just a **function** (Chapter 0!). It takes an input X and produces an output Y. The function has **knobs** inside it called *parameters* θ. "Learning" means **turning the knobs** until the function's outputs match the examples we were given.

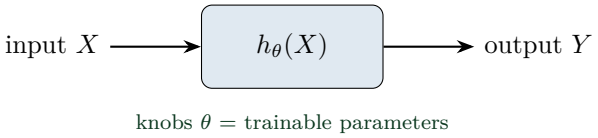


Figure — A model is a function h_θ with tunable knobs θ. Training = choosing θ so the outputs fit the examples. (This h is called the hypothesis.)

Concept

The course's running example (movie reviews). We are given reviews with a rating from -3 to $+3$, and we want to predict the rating of a *new* review. Simple model: give every word w a **sentiment score** θ_w, and predict a review's rating as the *sum of its words' scores*. Learning means finding good scores for all $\sim$ 100,000 English words — i.e. fitting a surface with 100,000 knobs.

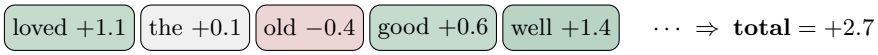


Figure — Add up the sentiment scores of the words in a review to predict its rating. Positive words push the score up, negative words pull it down — here the sum is +2.7.

Watch Out

This bag-of-words model ignores **word order** — "not good" and "good" get the same score. The lectures point this out: it is *not* how humans read. Real language models fix

this, but the simple sum is a perfect first example of "learning a function".

2.4 Measuring wrongness: the loss function

In Plain Words

To turn the knobs the *right* way, we need a score for "how wrong is the model right now". That score is the **loss** (or cost). Lower loss = better fit. Training is just "turn the knobs to make the loss as small as possible".

Concept

The course uses the **squared-error loss**. With training data $\{(x^{(i)}, y^{(i)}) : i = 1, \ldots, n\}$ and prediction $h_\theta(x^{(i)})$,

$$J(\theta) = \frac{1}{2n} \sum_{i=1}^{n} \left(h_\theta(x^{(i)}) - y^{(i)}\right)^2.$$

Each term $h_\theta(x^{(i)}) - y^{(i)}$ is one *residual* (how far the prediction is from the truth). We **square** them (so over- and under-shoots both count as positive and big misses hurt more), **sum** over all examples, and average. The $\frac{1}{2}$ is a convenience that will cancel a factor of 2 when we differentiate (Chapter 4).

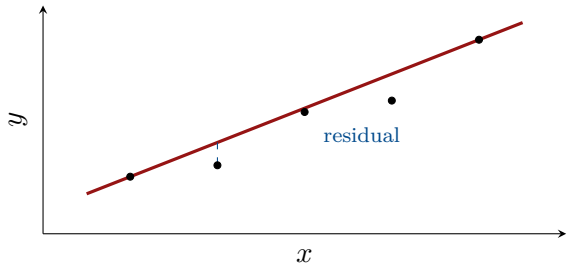


Figure — The loss adds up the squared lengths of the blue "residual" sticks. Training slides the red line to make that total as small as possible.

How to write it in Python

The loss is a direct translation of the formula $J = \frac{1}{2n}\sum(h_\theta(x^{(i)}) - y^{(i)})^2$:

```
import numpy as np
def loss(y_pred, y_true):
    n = len(y_true)
    return (1/(2*n)) * np.sum((y_pred - y_true)**2)   # J(theta)

y_true = np.array([1., 2., 3.])
y_pred = np.array([1.2, 1.8, 3.1])        # a slightly-off model's predictions
print(loss(y_pred, y_true))               # 0.015  (small = good fit)
```

`y_pred - y_true` is the residual vector; squaring, summing, and scaling by $\frac{1}{2n}$ gives the single cost number.

Analogy

The loss is a **golf score**: it measures how far you are from the hole, and you want it as *low* as possible. Each swing (knob-turn) that lowers your score is progress; training is playing until you can get no lower.

2.5 Training, testing, and not fooling yourself

In Plain Words

If you both study *and* write the exam from the same answer key, a high score proves nothing. So we **split the data**: train on most of it, then test on a part the model has never seen. The course uses about **80% for training, 20% for testing**.

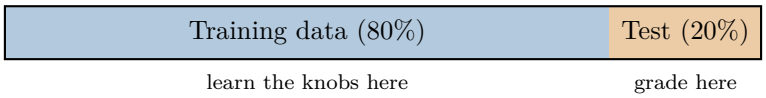


Figure — Train on 80%, then test on the untouched 20% to estimate how the model does on new data (its "generalisation").

Concept

Why not just try every function? Because of the *curse of dimensionality*: the lectures note that blindly learning a function of a high-dimensional input $X \in \mathbb{R}^n$ could need $\exp(n)$ examples — practically impossible. That is why we choose a *restricted* family of functions (like straight lines) and only tune their few knobs. Less freedom, but actually learnable.

2.6 How the knobs get turned: gradient descent (preview)

Concept

The training algorithm is **gradient descent**. Start with some guess $\theta^{(0)}$ and repeatedly step *downhill* on the loss:

$$\theta^{(t+1)} = \theta^{(t)} - \alpha \nabla J(\theta^{(t)}),$$

where α (the *learning rate*, e.g. 0.01) sets the step size and ∇J (the gradient, Chapter 0) points uphill — so we subtract it to go down. We devote all of Chapter 4 to this.

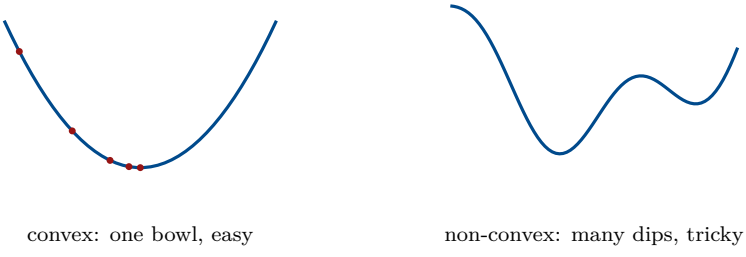
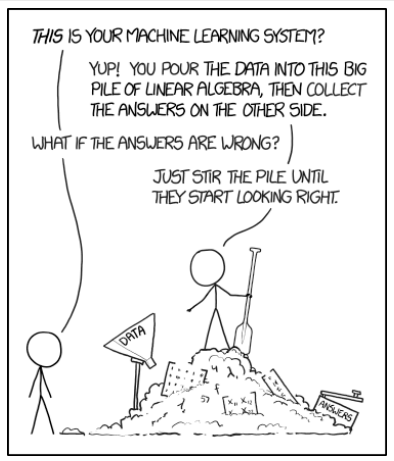


Figure — Gradient descent rolls downhill. A convex loss has a single bottom (linear regression); deep-learning losses are non-convex, with many local dips.

2.7 Where machine learning is used

The first lecture is a gallery of applications — all of them are "learn a function from examples" in disguise:

Application	The function being learned
Object detection	image → boxes + labels ("cat", "dog")
Face recognition / surveillance	face image → identity
Self-driving cars	sensor data → steering / braking
Image denoising & super-resolution	noisy/low-res image → clean/high-res image
Image colourisation	grayscale photo → colour photo
Emotion recognition	face → emotion label



xkcd #1838 "Machine Learning" by Randall Munroe — `https://xkcd.com/1838/` (CC BY-NC 2.5).

The joke: "pour your data into this big pile of linear algebra, then stir." This whole book is about opening up that pile — the "linear algebra" is the model h_θ, and the "stirring" is gradient descent minimising the loss.

Questions & Variations

Q. State Mitchell's definition of learning.

A. A program learns from experience E w.r.t. a task T and performance measure P if its performance at T, measured by P, improves with E.

Q. What are X, Y, θ, and h_θ?

A. X is the input, Y the output, θ the trainable parameters (knobs), and h_θ the model (hypothesis) mapping X to a prediction.

Q. What is a loss/cost function, and why square the errors?

A. A number measuring how wrong the model is; squaring makes all errors positive and penalises big misses more. The course uses $J(\theta) = \frac{1}{2n}\sum(h_\theta(x^{(i)}) - y^{(i)})^2$.

Q. Why split into training and test sets?

A. To measure how well the model does on *unseen* data (generalisation), rather than data it was fitted on.

Q. In one line, what does gradient descent do?

A. Repeatedly steps the parameters downhill on the loss: $\theta \leftarrow \theta - \alpha\nabla J$.

Chapter 3

Learning Paradigms

There is not one kind of machine learning — there are a few families, sorted by *what kind of data you have* and *what you want out*. This chapter maps them, using the course's own notation and examples.

3.1 The map of machine learning

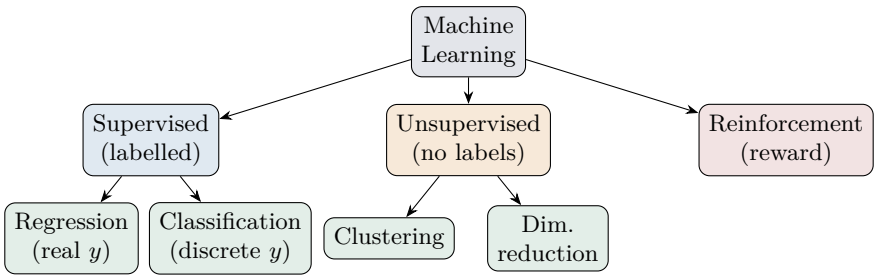


Figure — The three main families. Supervised learning splits by output type (real → regression, discrete → classification). Online, transfer and adversarial learning are extra flavours discussed at the end.

Analogy

Think of **three ways a child learns**. *Supervised*: a parent points and says "that's a dog, that's a cat" (labelled examples). *Unsupervised*: the child sorts a toy box into piles by themselves, with no one naming the piles (structure without labels). *Reinforcement*: the child touches a hot cup, gets "ouch" (a reward signal), and learns by trial and error.

3.2 The notation we will use

The course fixes this notation — worth memorising, because every later chapter uses it.

Symbol	Meaning
a, b, c	a scalar (a single number)
$\mathbf{x}, \mathbf{y}$	a vector (a list of numbers)
$\mathbf{A}, \mathbf{B}$	a matrix (a grid of numbers)
X, Y	a random variable
$a \in \mathcal{A}$	"a is a member of set $\mathcal{A}$"
$\lvert\mathcal{A}\rvert$	cardinality — how many items are in $\mathcal{A}$
$\lVert\mathbf{v}\rVert$	the norm (length) of vector $\mathbf{v}$
$\mathbf{u} \cdot \mathbf{v}$ or $\langle \mathbf{u}, \mathbf{v} \rangle$	dot product
$\mathbb{R}$, $\mathbb{R}^n$	the real numbers; n-dimensional real space
$f : \mathbb{R}^n \to \mathbb{R}$	a function taking an n-vector to one number
$x^{(i)}, y^{(i)}$	the i-th training input / output

The little superscript (i) always means "example number i", *not* a power. So $x^{(3)}$ is the 3rd data point, while x^3 is x cubed.

3.3 Supervised learning

In Plain Words

Supervised learning is learning *with an answer key.* You are given pairs of (input, correct output), and you learn a function that reproduces the outputs — so that when a *new* input arrives, you can predict its output.

Concept

Given labelled data $\{(x^{(1)}, y^{(1)}), \ldots, (x^{(n)}, y^{(n)})\}$, learn a mapping $h : x \mapsto y$ so that the prediction $\hat{y} = h_\theta(x)$ is close to the true y. The output y can be either **real-valued** (a number) or **discrete** (a label) — and that single choice splits supervised learning into two:

3.3.1 Regression — when the output is a number

In Plain Words

If the answer is a *quantity* — a price, a temperature, a house value — the task is **regression**. Regression = fitting a line (or a smooth curve) through the data so you can read off predictions.

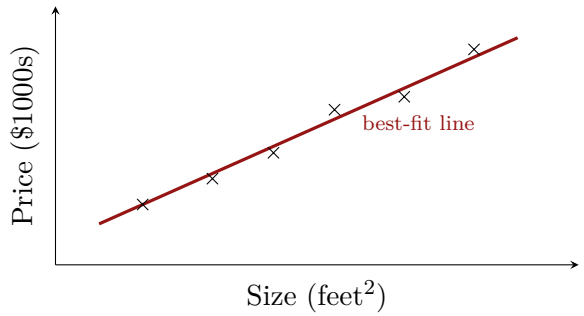
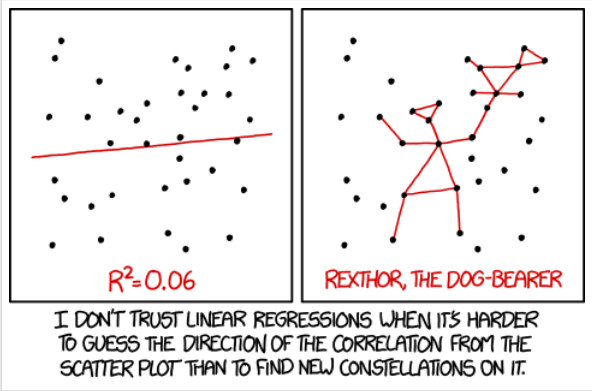


Figure — The course's housing example: each × is a house (size, price). Regression fits a line so you can predict the price of a new house from its size.



xkcd #1725 "Linear Regression" by Randall Munroe — https://xkcd.com/1725/ (CC BY-NC 2.5).

The joke: eyeball a cloud of points and draw a confident line. Regression is the honest, mathematical version of exactly that — and Chapter 3 shows precisely how the "best" line is chosen.

3.3.2 Classification — when the output is a label

In Plain Words

If the answer is a *category* — spam / not-spam, digit 0–9, tumour benign / malignant — the task is **classification**. Instead of a line to *sit on*, we look for a line (or curve) that *separates* the categories.

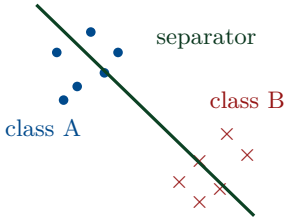


Figure — Classification finds a boundary that separates the classes (benign vs malignant, cat vs dog, digit 3 vs 8). The boundary can be a straight line or a curve.

Concept

Regression vs classification in one line: regression outputs a *continuous number* and fits a curve; classification outputs a *discrete label* and fits a separator. Same supervised setup, different kind of answer.

How to write it in Python

A linear classifier is just "compute a score, then check its sign". Points on one side of the boundary get label 1, the other side 0:

```
import numpy as np
def classify(X, w, b):
    score = X @ w + b              # a number for each point
    return np.where(score >= 0, 1, 0)      # sign -> class label

X = np.array([[1., 3], [2, 2.5], [3, 1], [3.5, 0.5]])
print(classify(X, w=np.array([1., -1]), b=-0.5))    # [0 0 1 1]
```

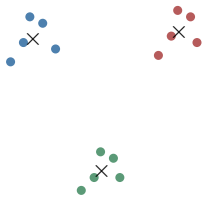
The line $w \cdot x + b = 0$ is the separator; `np.where` turns the score's sign into a 0/1 label. (Learning w, b is what logistic regression / SVMs do later in the course.)

3.4 Unsupervised learning

In Plain Words

Unsupervised learning has *no answer key* — only inputs $\{x^{(1)}, \ldots, x^{(n)}\}$, no y's. The goal is to discover *hidden structure*: which points group together, or what few directions explain most of the data.

3.4.1 Clustering — finding natural groups



three clusters, × = centre of each

Figure — Clustering (e.g. K-Means) sorts unlabelled points into groups by closeness — no one told it the groups; it found them. Used to cluster images or documents.

How to write it in Python

The heart of K-Means is "assign each point to its nearest centre". One assignment step:

```
import numpy as np
pts     = np.array([[0., 0], [0.2, 0.1], [5, 5], [5.1, 4.9]])
centres = np.array([[0., 0], [5., 5]])
# distance from every point to every centre, then pick the closest
dist   = np.linalg.norm(pts[:, None, :] - centres[None, :, :], axis
    =2)
labels = dist.argmin(axis=1)
print(labels)          # [0 0 1 1]  -> first two join centre 0, last
     two centre 1
```

Full K-Means just repeats: assign points to nearest centre, then move each centre to the average of its points, until nothing changes.

3.4.2 Dimensionality reduction — squashing without losing much

In Plain Words

Sometimes data has many columns but really only varies along a few directions. Dimensionality reduction (like PCA) finds those directions and *projects* the data down — fewer numbers, almost the same information.

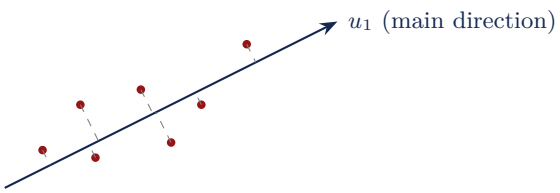


Figure — Every point is dropped (dashed) onto the main direction u_1. Keeping just its position along u_1 replaces two numbers with one, losing little.

3.4.3 A peek at topic modelling

Concept

Given many documents with no labels, *topic modelling* (LDA) discovers themes on its own. In the course example, three sentences about matches, elections and teams get sorted into two topics — "Sports" {game, players, score, match} and "Politics" {election, results, policy} — and each document becomes a *mixture*, e.g. 80% Sports / 20% Politics. Two ideas: each document is a mixture of topics, and each topic is a probability distribution over words.

3.5 Reinforcement learning

In Plain Words

Reinforcement learning has no fixed dataset at all. An **agent** acts in an **environment**, receives a **reward** (good or bad), and learns by *trial and error* which actions earn the most reward over time.

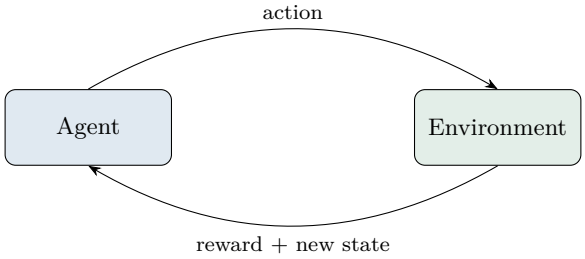


Figure — The reinforcement loop: the agent takes an action, the environment returns a reward and a new state, and the agent adjusts to earn more reward. Like a baby learning to walk: try, wobble, reward, improve.

3.6 Other flavours (briefly)

- **Online learning** — data arrives one example (or a small batch) at a time, and the model updates on the fly instead of from a fixed dataset.
- **Transfer learning** — knowledge learned on one task boosts a related task (knowing how to ride a bicycle helps you learn a motorbike).
- **Adversarial ML** — a tiny, cleverly-chosen change to the input can fool a model. The famous course example: adding an imperceptible noise pattern to a *panda* photo makes a classifier call it a *gibbon* with 99% confidence.

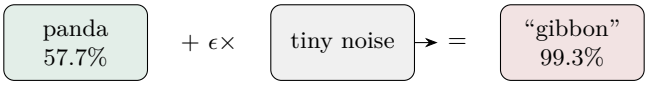


Figure — Adversarial example: a panda plus a noise pattern too faint for us to see flips the model's answer to "gibbon". A reminder that models can be brittle.

Questions & Variations

Q. What is the difference between supervised and unsupervised learning?
A. Supervised uses labelled data (inputs *and* correct outputs) to learn $x \to y$; unsupervised uses only inputs and finds hidden structure (groups, directions).

Q. Regression vs classification?
A. Both are supervised. Regression predicts a continuous number (fit a curve); classification predicts a discrete label (fit a separator).

Q. Give one example each of regression and classification.
A. Regression: predict house price from size. Classification: decide if a tumour is benign or malignant.

Q. What does clustering do, and name one algorithm.
A. Groups unlabelled points by similarity; K-Means is a standard algorithm.

Q. What defines reinforcement learning?
A. An agent learns by interacting with an environment and maximising a reward signal through trial and error — no fixed labelled dataset.

Q. What does the superscript (i) mean in $x^{(i)}$?
A. It indexes the i-th training example — not a power.

Chapter 4

Linear Regression

Our first real model. Linear regression is supervised, it does regression (real-valued output), and it is a straight line. Everything in this chapter is built from Chapter 0's tools: a line, a slope, and the sum $\sum$.

4.1 The model: a straight line

In Plain Words

Linear regression predicts the output as a **straight-line function** of the input: *output* = *intercept* + *slope* × *input*. The whole job of "learning" is to pick the best intercept and slope so the line sits nicely through the data.

Concept

For one input feature x, the model (the *hypothesis*) is

$$h_\theta(x) = \theta_0 + \theta_1 x,$$

where θ_0 is the intercept and θ_1 the slope (exactly the $y = mx + c$ line from Chapter 0, renamed). The θ's are the knobs we tune.

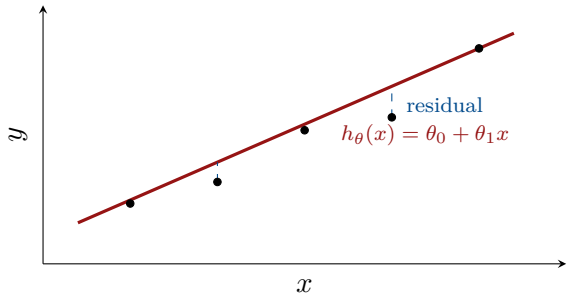


Figure — The model is a line. A residual (blue) is the vertical gap between a data point and the line — the model's error on that point.

4.2 More than one feature: the general hypothesis

Concept

With d features $x_1, \ldots, x_d$, the line becomes a weighted sum:

$$h_\theta(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_d x_d = \sum_{j=0}^{d} \theta_j x_j,$$

where we sneak in a constant feature $x_0 = 1$ so the intercept θ_0 joins the sum tidily (it multiplies $x_0 = 1$). Then, with the vectors $x = [x_0, \ldots, x_d]$ and $\theta = [\theta_0, \ldots, \theta_d]^\top$, the whole thing is just a **dot product** (Chapter 0): $h_\theta(x) = \theta^\top x$. Stacking all n examples into a matrix X, all predictions at once are $X\theta$.

Analogy

The bias trick ($x_0 = 1$) is like adding a "base fare" to a rickshaw meter. The fare is *base* + *rate* × distance. Writing the base as "rate ×1" lets you treat it like any other term — one clean sum instead of a special case.

4.2.1 The matrix form: all predictions at once

In Plain Words

Writing one prediction is fine, but we have n examples. Instead of a loop, we stack all the data into a **design matrix** X and get *every* prediction in a single multiply. This is the "vectorization" idea — and it is why real ML code is fast.

Concept

Put a 1 (for the bias) and the features of example i in row i of X, and stack the targets in y. Then all predictions are one matrix–vector product, and the cost is one squared length:

$$\hat{y} = X\theta, \qquad J(\theta) = \frac{1}{2n}\|X\theta - y\|_2^2 = \frac{1}{2n}(X\theta - y)^\top (X\theta - y).$$

Here $\|v\|_2^2 = v^\top v = \sum_i v_i^2$ (Chapter 0) is just "sum of squares" — so this matrix formula is *exactly* the same $\frac{1}{2n}\sum(h_\theta(x^{(i)}) - y^{(i)})^2$ as before, only written compactly.

$$\underbrace{\begin{bmatrix} 1 & x^{(1)} \\ 1 & x^{(2)} \\ 1 & x^{(3)} \end{bmatrix}}_{X} \underbrace{\begin{bmatrix} \theta_0 \\ \theta_1 \end{bmatrix}}_{\theta} = \underbrace{\begin{bmatrix} \theta_0 + \theta_1 x^{(1)} \\ \theta_0 + \theta_1 x^{(2)} \\ \theta_0 + \theta_1 x^{(3)} \end{bmatrix}}_{\hat{y}} - \underbrace{\begin{bmatrix} y^{(1)} \\ y^{(2)} \\ y^{(3)} \end{bmatrix}}_{y} = \text{residuals}$$

Figure — The design matrix in action: $X\theta$ produces all predictions at once; subtracting y gives the residual vector, whose squared length (over $2n$) is the cost.

How to write it in Python

Predicting and scoring the line is two short lines of NumPy. Using the notes' example data $(1,1), (2,2), (3,3)$:

```python
import numpy as np
x = np.array([1., 2., 3.]);  y = np.array([1., 2., 3.])
X = np.column_stack([np.ones(len(x)), x])       # design matrix:
    column of 1s + feature

def predict(theta):  return X @ theta            # y_hat = X theta
def cost(theta):
    m = len(y)
    return (1/(2*m)) * np.sum((predict(theta) - y)**2)    # J(theta)

print(cost(np.array([0, 1.0])))     # perfect fit   -> 0.0
print(cost(np.array([0, 0.5])))     # too shallow   -> 0.5833
```

The maths $X\theta$ becomes `X @ theta`; the squared-error sum becomes `np.sum((...)**2)`. Same formula, now runnable.

4.3 What makes a line "good"? The cost function

In Plain Words

A line is good if its predictions are close to the real answers — i.e. the residuals are small. We add up the *squared* residuals to get one "badness" number, the **cost**. Small cost = good line. Training = making the cost as small as possible.

Concept

The course uses the **Least-Squares Cost Function**:

$$J(\theta) = \frac{1}{2n} \sum_{i=1}^{n} \left(h_\theta(x^{(i)}) - y^{(i)} \right)^2.$$

- $h_\theta(x^{(i)}) - y^{(i)}$ is the residual on example i (prediction minus truth).
- We **square** it: negatives and positives both count, and big misses are punished harder.
- We **sum** over all n examples and divide by n (an average), and the extra $\frac{1}{2}$ is a convenience — it will cancel the "2" that appears when we differentiate the square in Chapter 4.

Remember: the superscript (i) means "example i", not a power.

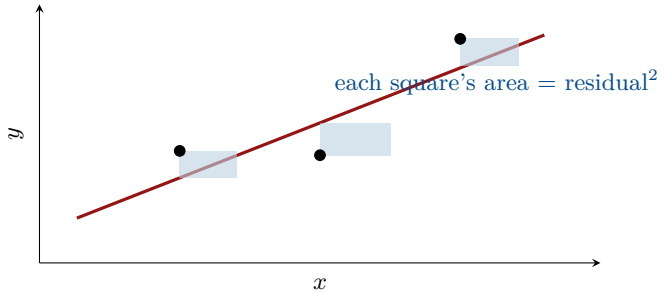


Figure — Why it is called "squared" error: each residual is the side of a literal square, and the cost adds up their areas. A point twice as far off contributes four times the area — that is why big misses dominate.

4.4 Worked example: computing the cost

In Plain Words

Let us actually turn the crank on tiny data — the exact example from the class notes — so the formula stops being scary.

Derivation — step by step

Data: three points $(1,1),(2,2),(3,3)$, so $n=3$. Take $\theta_0=0$, so $h_\theta(x)=\theta_1 x$. Then

$$J(\theta_1)=\frac{1}{2\cdot 3}\sum_{i=1}^{3}\left(\theta_1 x^{(i)}-y^{(i)}\right)^2.$$

Try $\theta_1=1$: the line $y=x$ hits all three points, residuals are $0,0,0$, so $J(1)=0$. Perfect fit.
Try $\theta_1=0.5$: predictions are $0.5,1,1.5$; residuals $0.5-1,\ 1-2,\ 1.5-3=-0.5,-1,-1.5$. Square and add:

$$J(0.5)=\frac{1}{6}\left[(-0.5)^2+(-1)^2+(-1.5)^2\right]=\frac{1}{6}\left[0.25+1+2.25\right]=\frac{3.5}{6}\approx 0.58.$$

Try $\theta_1=1.5$: by the same steps, $J(1.5)=\frac{1}{6}[0.25+1+2.25]\approx 0.58$ — the *same* as $\theta_1=0.5$, because it is equally wrong on the other side.

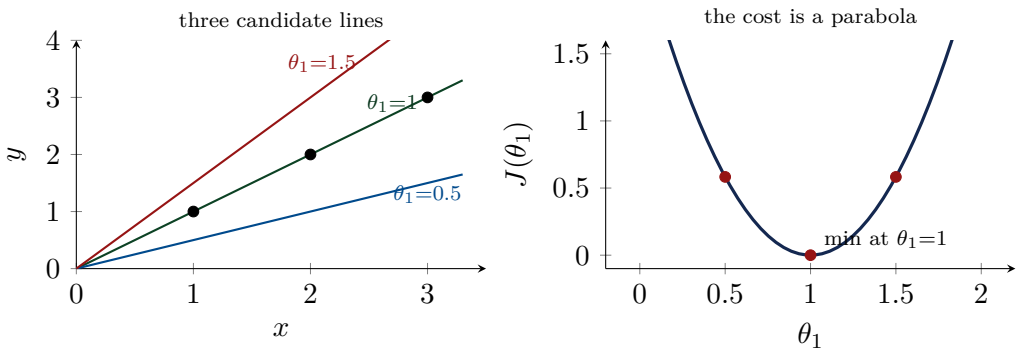


Figure — Left: the three lines against the data. Right: plotting the cost for every θ_1 traces a ***parabola*** *with its bottom at $\theta_1=1$ (cost 0) — and $\theta_1=0.5, 1.5$ sit equally high at 0.58.*

4.5 The shape of the cost: a convex bowl

Concept

With *both* knobs free, $J(\theta_0,\theta_1)$ is a surface over the two parameters. For linear regression it is a **convex bowl** — one single lowest point, no other dips. Seen from above, its level sets are **concentric ellipses** (contours); the centre of the ellipses is the best (θ_0,θ_1). Convexity is the reason gradient descent is guaranteed to find the one true minimum here (unlike the wavy surfaces of deep learning).

Figure — The least-squares cost is a convex bowl. Each contour ellipse is one "height" of cost; every choice of (θ_0, θ_1) is a point, and the best line sits at the centre.

4.6 Finding the bottom of the bowl

In Plain Words

So training linear regression means: *find the (θ_0, θ_1) at the bottom of the bowl.* We do it the way the notes describe — pick a starting guess, then repeatedly nudge θ in the direction that lowers J. That downhill procedure is **gradient descent**, and it gets its own chapter next.

Questions & Variations

Q. Write the linear-regression hypothesis for one feature and for many.
A. One feature: $h_\theta(x) = \theta_0 + \theta_1 x$. Many: $h_\theta(x) = \sum_{j=0}^{d} \theta_j x_j = \theta^\top x$, with $x_0 = 1$.

Q. What is the least-squares cost function?
A. $J(\theta) = \frac{1}{2n} \sum_{i=1}^{n} (h_\theta(x^{(i)}) - y^{(i)})^2$ — the average squared residual (with a $\frac{1}{2}$).

Q. Why is there a $\frac{1}{2}$?
A. Convenience: it cancels the factor 2 produced when differentiating the square, giving a cleaner gradient.

Q. For data $(1,1), (2,2), (3,3)$ and $h = \theta_1 x$, what is J at $\theta_1 = 1$ and $\theta_1 = 0.5$?
A. $J(1) = 0$ (perfect fit); $J(0.5) = \frac{1}{6}[0.25 + 1 + 2.25] \approx 0.58$.

Q. What shape is the least-squares cost, and why does it matter?
A. A convex bowl with a single minimum, so gradient descent reliably finds the global best.

Chapter 5

Gradient Descent

Chapter 3 ended with a bowl and a question: how do we find its bottom? Gradient descent is the answer — the workhorse algorithm that trains almost every model in this course.

5.1 The idea: roll downhill

In Plain Words

Imagine a ball placed on the inside of the cost bowl. Let go, and it rolls *downhill* until it settles at the bottom. Gradient descent is that ball, done with maths: from wherever you are, figure out which way is downhill, take a small step that way, and repeat.

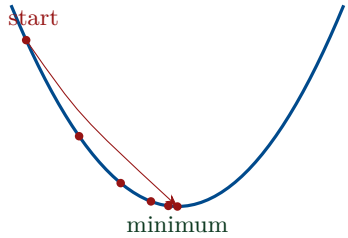


Figure — Gradient descent rolls the parameter downhill on the cost curve in small steps, until it reaches the bottom (the best parameters).

5.2 The update rule

Concept

For each parameter θ_j, one step of gradient descent is

$$\theta_j \;:=\; \theta_j - \alpha \,\frac{\partial J(\theta)}{\partial \theta_j}.$$

- $\frac{\partial J}{\partial \theta_j}$ is the slope of the cost in the θ_j direction — it points *uphill*.
- We **subtract** it, so we move *downhill*.
- α (the **learning rate**) is the step size — a small positive number like 0.01. The symbol := means "update to".

Watch Out

Simultaneous update. With two knobs, compute *both* new values from the *old* θ first, then assign:

$$t_0 := \theta_0 - \alpha\,\partial_{\theta_0} J, \quad t_1 := \theta_1 - \alpha\,\partial_{\theta_1} J, \quad \text{then } \theta_0 := t_0,\ \theta_1 := t_1.$$

If you overwrite θ_0 *before* computing t_1, the new θ_0 leaks into the θ_1 gradient — that is the *wrong* way, and it changes the algorithm. Always use the old values for both, then update together.

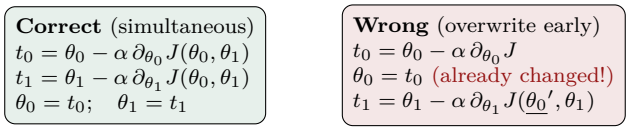


Figure — The notes' warning, drawn: on the left both gradients use the old θ. On the right, θ_0 is overwritten first, so the new θ_0' leaks into the θ_1 gradient — a different, incorrect algorithm.

5.3 Why subtract the gradient? The sign-of-slope trick

In Plain Words

Subtracting the slope automatically moves you the right way, on *either* side of the valley. If you are on the right slope (going uphill to the right, positive slope), subtracting it moves you *left* toward the bottom. If you are on the left slope (negative slope), subtracting a negative *adds*, moving you *right* toward the bottom. Either way, you head for the minimum.

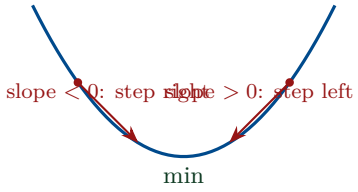


Figure — The update $\theta := \theta - \alpha \cdot \text{slope}$ always points toward the minimum: positive slope ⇒ move left, negative slope ⇒ move right.

5.4 Reminder: the derivative, checked with numbers

Derivation — step by step

The gradient is made of derivatives (slopes), and we can *see* the power rule with numbers (this is exactly how the lecture motivates it).
$f(a) = 3a$**:** at $a = 2$, $f = 6$; at $a = 2.001$, $f = 6.003$. Slope $= \frac{6.003-6}{0.001} = \frac{0.003}{0.001} = 3$. Try $a = 5$: same slope 3. So $f'(a) = 3$ (a constant), matching the power/constant rule.
$f(a) = a^2$**:** at $a = 2$, $f = 4$; at $a = 2.001$, $f \approx 4.004$. Slope $= \frac{0.004}{0.001} = 4 = 2 \cdot 2$. At $a = 5$: slope $\approx \frac{0.010}{0.001} = 10 = 2 \cdot 5$. So $f'(a) = 2a$ — the power rule, confirmed by arithmetic.

5.5 The star derivation: the gradient of the cost

In Plain Words

To run gradient descent on linear regression, we need $\partial J/\partial\theta_j$. Here is the full derivation — every step spelled out.

Derivation — step by step

Start from the cost and differentiate with respect to one knob θ_j:

$$\frac{\partial}{\partial\theta_j}J(\theta) = \frac{\partial}{\partial\theta_j}\,\frac{1}{2n}\sum_{i=1}^{n}\big(h_\theta(x^{(i)}) - y^{(i)}\big)^2.$$

Step 1 — write out the prediction $h_\theta(x^{(i)}) = \sum_{k=0}^{d}\theta_k x_k^{(i)}$:

$$= \frac{1}{2n}\sum_{i=1}^{n}\frac{\partial}{\partial\theta_j}\Big(\sum_{k=0}^{d}\theta_k x_k^{(i)} - y^{(i)}\Big)^2.$$

Step 2 — chain rule (bring the power 2 down, times the derivative of the inside):

$$= \frac{1}{2n}\sum_{i=1}^{n}2\Big(\sum_{k}\theta_k x_k^{(i)} - y^{(i)}\Big)\cdot\frac{\partial}{\partial\theta_j}\Big(\sum_{k}\theta_k x_k^{(i)} - y^{(i)}\Big).$$

Step 3 — the inner derivative. In $\sum_k \theta_k x_k^{(i)}$, only the $k = j$ term contains θ_j; its derivative is $x_j^{(i)}$. Everything else (including $y^{(i)}$) is constant, derivative 0. So the inner derivative is $x_j^{(i)}$.

$$= \frac{1}{2n}\sum_{i=1}^{n}2\big(h_\theta(x^{(i)}) - y^{(i)}\big)\,x_j^{(i)}.$$

Step 4 — the 2 cancels the $\frac{1}{2}$:

$$\boxed{\frac{\partial}{\partial\theta_j}J(\theta) = \frac{1}{n}\sum_{i=1}^{n}\big(h_\theta(x^{(i)}) - y^{(i)}\big)\,x_j^{(i)}}.$$

Concept

Plugging this into the update rule gives the **linear-regression gradient-descent update**:

$$\theta_j := \theta_j - \alpha\,\frac{1}{n}\sum_{i=1}^{n}\big(h_\theta(x^{(i)}) - y^{(i)}\big)\,x_j^{(i)}.$$

In words: each error $(h - y)$ is multiplied by the feature x_j that caused it, averaged over all examples, and used to nudge θ_j. *This* is why the $\frac{1}{2}$ was in the cost — so this update comes out clean, with no stray 2.

How to write it in Python

Gradient descent is a short loop. First, the exact worked example from the notes ($J = \theta_1^2 + \theta_2^2 + 4$, start $(3, 4)$):

```
import numpy as np
def grad(t):  return np.array([2*t[0], 2*t[1]])      # gradient of J

theta = np.array([3., 4.])
for step in range(30):
    theta = theta - 0.1 * grad(theta)                # alpha = 0.1 (slow case)
print(theta)      # -> [0.0037  0.005]  (creeping toward the minimum (0,0))
```

Change `0.1` to `0.5` and it lands on $(0, 0)$ in a *single* step; change it to `2` and it blows up — exactly the four cases below. Now the same loop for **linear regression**, using the gradient we just derived:

```
def gradient_descent(X, y, alpha=0.1, epochs=500):
    theta = np.zeros(X.shape[1]);  m = len(y)
    for _ in range(epochs):
        grad = (1/m) * (X.T @ (X @ theta - y))       # (1/n) sum (h - y) x_j
        theta = theta - alpha * grad                 # the update rule
    return theta
```

The maths line $\frac{1}{n}\sum(h_\theta(x^{(i)}) - y^{(i)})x_j^{(i)}$ is literally `(1/m)*(X.T @ (X @ theta - y))` — prediction, minus target, folded back through $X^\top$.

5.6 The learning rate α: four behaviours

In Plain Words

The step size α makes or breaks gradient descent. Too big and you overshoot; too small and you crawl. The class runs one tiny example at four values of α to show all the cases — let us reproduce it exactly.

Derivation — step by step

Setup: $J(\theta) = \theta_1^2 + \theta_2^2 + 4$, so $\partial_{\theta_1} J = 2\theta_1$ and $\partial_{\theta_2} J = 2\theta_2$. The bowl's minimum is at $(0, 0)$ with $J_{\min} = 4$. Start at $(\theta_1, \theta_2) = (3, 4)$, where $\nabla J = (6, 8)$ and $J = 29$. Each step multiplies θ by $(1 - 2\alpha)$.

α	Factor $(1 - 2\alpha)$	What happens	Behaviour
2	-3	$(3,4) \to (-9,-12) \to (27,36)$, $J : 29 \to 229 \to 2029$	**diverges**
1	-1	$(3,4) \leftrightarrow (-3,-4)$, J stuck at 29	**oscillates**
0.1	0.8	$(3,4) \to (2.4,3.2) \to \ldots \to (0,0)$, $J \to 4$ (~30 steps)	**slow**
0.5	0	$(3,4) \to (0,0)$ in *one* step, $J \to 4$	**exact**

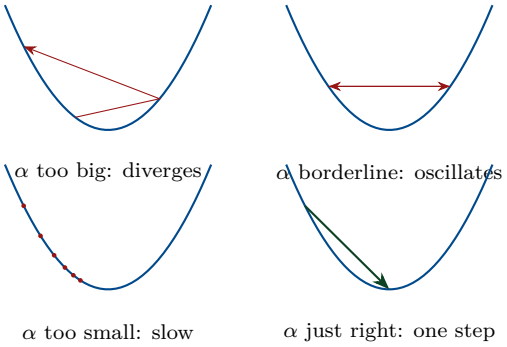


Figure — The four learning-rate cases from the worked example. Big α overshoots outward (diverge); the borderline value ping-pongs; small α inches in; the right α lands at the bottom (here, in a single step because the bowl is a simple quadratic).

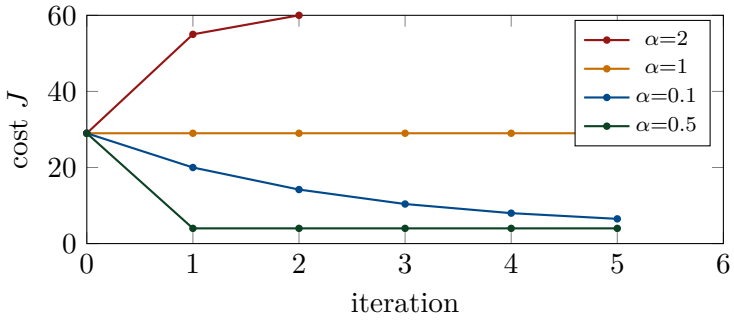


Figure — Cost vs iteration for the four rates: α=2 shoots up (diverges), α=1 stays flat (oscillates), α=0.1 eases down (slow), α=0.5 drops to the minimum immediately.

Chai-Break

Choosing α is like adjusting the shower tap. A tiny turn and you wait forever for warm water (too small). A big turn and you scald then freeze, back and forth (too big / oscillating). The right touch gets you comfortable fast.

5.7 Seeing gradient descent in 2-D and 3-D

In Plain Words

With two knobs, the cost is a bowl-shaped surface. Seen from above, its "height lines" are the contour ellipses; the gradient at any point is an arrow pointing straight *uphill* (perpendicular to the contour). Gradient descent walks the opposite way — inward, toward the centre.

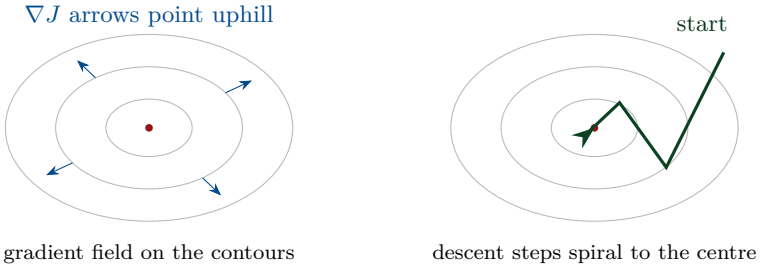


Figure — Left: the gradient field — every arrow points uphill, at right angles to the contour. Right: gradient descent steps against those arrows, spiralling inward to the minimum (the housing-example trajectory from the lectures).

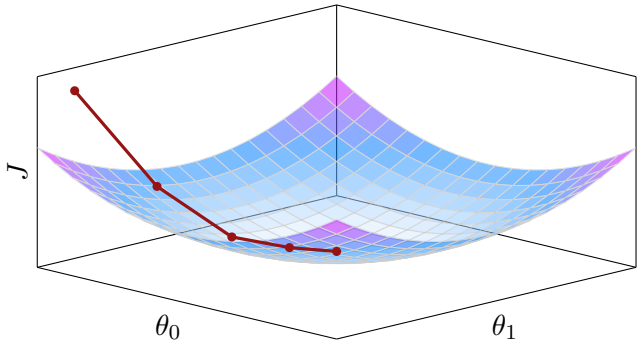


Figure — The same thing in 3-D: the cost is a bowl over the two parameters, and gradient descent (red) walks down its inside wall to the bottom.

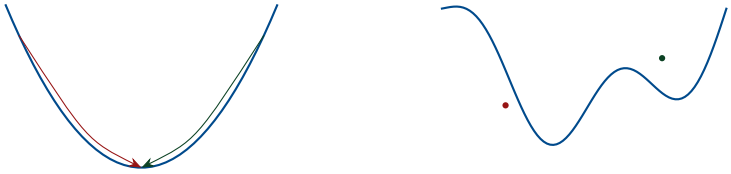


Figure — Why convexity matters (Lec 5): linear regression's bowl is convex, so any starting point reaches the one global minimum. A wavy (non-convex) loss has many local dips, and the starting point decides which one you land in.

5.8 When to stop

Concept

Gradient descent runs "until convergence". In practice you stop when any of these is below a small tolerance ϵ (from the notes):

$$\text{(i) } \|\theta^{(k+1)} - \theta^{(k)}\| \le \epsilon, \quad \text{(ii) } \|\nabla J(\theta^{(k)})\| \le \epsilon, \quad \text{(iii) } |J(\theta^{(k+1)}) - J(\theta^{(k)})| \le \epsilon.$$

That is: the parameters stop moving, or the slope is nearly flat, or the cost stops improving. (Here $\|\cdot\|$ is the vector length from Chapter 0, e.g. $\|(2,3,4)\| = \sqrt{4+9+16} = \sqrt{29}$.)

Questions & Variations

Q. Write the gradient-descent update rule and name each part.
A. $\theta_j := \theta_j - \alpha\, \partial J/\partial \theta_j$: subtract the uphill slope, scaled by the learning rate α, to move downhill.

Q. Derive $\partial J/\partial \theta_j$ for linear regression.

A. Chain rule on $\frac{1}{2n}\sum(h-y)^2$: the 2 comes down, the inner derivative is $x_j^{(i)}$, the 2 cancels the $\frac{1}{2}$, giving $\frac{1}{n}\sum(h_\theta(x^{(i)}) - y^{(i)})x_j^{(i)}$.

Q. What is the simultaneous-update rule and why does it matter?
A. Compute all new θ's from the old values, then assign together; overwriting one early

leaks it into the others' gradients and changes the algorithm.

Q. What happens for too-large vs too-small α?

A. Too large: overshoot, oscillate, or diverge. Too small: converges but very slowly.

Q. Give one stopping criterion.

A. Stop when $\|\nabla J\| \leq \epsilon$ (gradient near zero) — or when θ or J barely changes.

Chapter 6

Gradients and Hessians in Full

Chapter 0 met the derivative; Chapter 4 used the gradient. Now we build the full toolkit the course needs: gradients and Hessians of vectors and matrices, and the beautiful result they lead to — the least-squares normal equation.

6.1 The gradient of a function of many variables

Concept

For a function $f : \mathbb{R}^n \to \mathbb{R}$ (many inputs, one number out), the **gradient** stacks all the partial derivatives into a vector:

$$\nabla_x f(x) = \left[\frac{\partial f}{\partial x_1},\ \frac{\partial f}{\partial x_2},\ \ldots,\ \frac{\partial f}{\partial x_n}\right]^\top.$$

If the input is a whole *matrix* A, the gradient is a matrix of the same shape, with $\big(\nabla_A f\big)_{ij} = \partial f / \partial A_{ij}$. Two rules we lean on: *linearity* $\nabla(f+g) = \nabla f + \nabla g$ and $\nabla(t\,f) = t\,\nabla f$. And a caution from the notes: $\nabla_x(Ax)$ is **not defined** — the gradient is only for functions that output a single number, but Ax outputs a vector.

$$f(x_1, x_2, x_3) \xrightarrow{\text{one slope per input}} \nabla f = \begin{bmatrix} \partial f/\partial x_1 \\ \partial f/\partial x_2 \\ \partial f/\partial x_3 \end{bmatrix}$$

Figure — The gradient is just "the slope in every direction, stacked into a vector".

6.2 Why the gradient points the steepest way (a proof)

Derivation — step by step

Claim: ∇J points in the direction of fastest increase. **Proof (from the notes):** the rate of change of J in a unit direction $\vec{v}$ is the dot product

$$\frac{\partial J}{\partial \vec{v}} = \nabla J \cdot \vec{v} = \|\nabla J\|\,\|\vec{v}\| \cos\theta,$$

where θ is the angle between $\vec{v}$ and ∇J. Since $\|\nabla J\|$ is fixed and $\|\vec{v}\| = 1$, this is largest exactly when $\cos\theta = 1$, i.e. $\theta = 0$ — when $\vec{v}$ points *along* ∇J. So the gradient is the steepest-ascent direction, and $-\nabla J$ (used in gradient descent) is steepest *descent*. ■

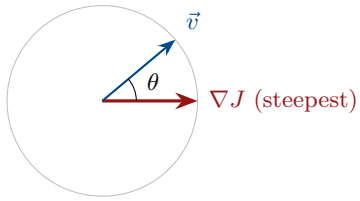


Figure — Moving along $\vec{v}$ climbs at rate $\|\nabla J\| \cos\theta$. The climb is fastest when $\vec{v}$ lines up with ∇J ($\theta = 0$).

6.3 The Hessian: second derivatives in a matrix

Concept

The **Hessian** $H = \nabla_x^2 f$ collects all the *second* partial derivatives: $H_{ij} = \dfrac{\partial^2 f}{\partial x_i \, \partial x_j}$. It is always **symmetric** ($H_{ij} = H_{ji}$) because the order of differentiation does not matter. Where the gradient gave slope, the Hessian gives *curvature* in every pair of directions.

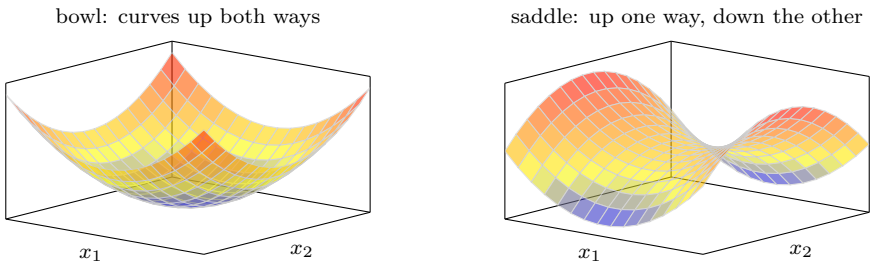


*Figure — Curvature made visible. A **bowl** ($z = x_1^2 + x_2^2$) curves up in every direction — Hessian eigenvalues all positive (a minimum). A **saddle** ($z = x_1^2 - x_2^2$) curves up one way and down the other — mixed-sign eigenvalues.*

Watch Out

The Hessian is NOT "the gradient of the gradient" written naively. The gradient ∇f is a *vector* (outputs several numbers), and ∇ of a vector is not defined. The correct way (from the notes): take the gradient of *each component* $\partial f / \partial x_i$; that gives the i-th *column* of the Hessian. Assemble the columns:

$$\nabla_x^2 f(x) = \nabla_x \big(\nabla_x f(x)\big)^\top.$$

This is exactly the "build it column by column" idea used in the Lab 2 numerical Hessian.

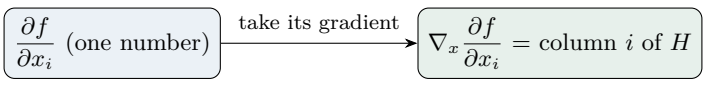


Figure — Correct Hessian construction: differentiate each gradient component again to get one column of H; stack the columns.

6.4 Gradient & Hessian of a linear function

Derivation — step by step

Let $f(x) = b^\top x = \sum_{i=1}^{n} b_i x_i$ (a weighted sum). Differentiate w.r.t. x_k: only the $i = k$ term has x_k, and its derivative is b_k. So

$$\frac{\partial f}{\partial x_k} = b_k \quad \Longrightarrow \quad \boxed{\nabla_x \, b^\top x = b}.$$

This is the vector version of the scalar fact $\frac{d}{dx}(ax) = a$. Since the gradient b is a constant, all second derivatives vanish: the **Hessian of a linear function is** 0.

6.5 Gradient & Hessian of a quadratic function

In Plain Words

Quadratics are the important case — our cost bowls *are* quadratics. The key result: for a symmetric matrix A, the function $f(x) = x^\top Ax$ has gradient $2Ax$ and Hessian $2A$. This is the exact vector version of the scalar $\frac{d}{dx}(ax^2) = 2ax$ and $\frac{d^2}{dx^2}(ax^2) = 2a$.

Derivation — step by step

See it on a 2×2 example. Let $A = \begin{bmatrix} a & b \\ b & c \end{bmatrix}$ (symmetric). Then

$$x^\top Ax = ax_1^2 + 2b\,x_1x_2 + c\,x_2^2.$$

Differentiate: $\partial/\partial x_1 = 2ax_1 + 2bx_2$ and $\partial/\partial x_2 = 2bx_1 + 2cx_2$. Stack them:

$$\nabla f = \begin{bmatrix} 2ax_1 + 2bx_2 \\ 2bx_1 + 2cx_2 \end{bmatrix} = 2 \begin{bmatrix} a & b \\ b & c \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = 2Ax.$$

Differentiating once more gives the constant matrix $\begin{bmatrix} 2a & 2b \\ 2b & 2c \end{bmatrix} = 2A$ — the Hessian.

Derivation — step by step

The general $n \times n$ proof (the full notes derivation). Write the quadratic as a double sum and differentiate with respect to one variable x_k:

$$f(x) = x^\top Ax = \sum_{i=1}^{n}\sum_{j=1}^{n} A_{ij}\, x_i x_j, \qquad \frac{\partial f}{\partial x_k} = \frac{\partial}{\partial x_k} \sum_i \sum_j A_{ij} x_i x_j.$$

Which terms survive? A term $A_{ij}x_ix_j$ depends on x_k only if $i = k$ or $j = k$. Splitting those out:

$$\frac{\partial f}{\partial x_k} = \underbrace{\sum_{i \neq k} A_{ik}x_i}_{\text{from } j=k} + \underbrace{\sum_{j \neq k} A_{kj}x_j}_{\text{from } i=k} + \underbrace{2A_{kk}x_k}_{\text{from } A_{kk}x_k^2}.$$

Fold the $2A_{kk}x_k$ back into the two sums (it supplies the missing $i = k$ and $j = k$

terms), so each sum becomes a full sum:

$$= \sum_{i=1}^{n} A_{ik}x_i + \sum_{j=1}^{n} A_{kj}x_j.$$

Rename the dummy index $j \to i$ in the second sum, then use **symmetry** $A_{ik} = A_{ki}$:

$$= \sum_{i} A_{ik}x_i + \sum_{i} A_{ki}x_i = 2\sum_{i} A_{ki}x_i = (2Ax)_k.$$

Since this holds for every k, $\nabla_x\, x^\top Ax = 2Ax$. ■

Vector/matrix form	Scalar analogy	Hessian
$\nabla_x\, b^\top x = b$	$\frac{d}{dx}(ax) = a$	0
$\nabla_x\, x^\top Ax = 2Ax$ (sym. A)	$\frac{d}{dx}(ax^2) = 2ax$	$2A$

6.6 Least squares: the normal equation

In Plain Words

Here is the payoff. Suppose we want $Ax = b$ but no exact solution exists (there are more equations than unknowns, so b is "out of reach"). Least squares finds the x that makes Ax *as close as possible* to b — the foot of the perpendicular from b onto the set of reachable points.

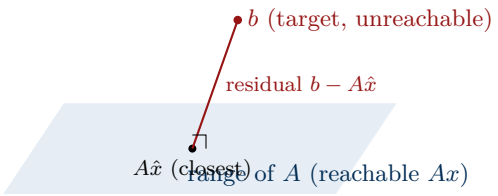


Figure — Least squares: b sticks out of the plane of reachable vectors. The best $A\hat{x}$ is the point on the plane closest to b; the leftover residual is perpendicular to the plane.

Derivation — step by step

Minimise the squared distance $\|Ax - b\|_2^2$. Using $\|v\|_2^2 = v^\top v$ and expanding:

$$\begin{aligned}\|Ax - b\|_2^2 &= (Ax - b)^\top (Ax - b) \\ &= x^\top A^\top Ax - x^\top A^\top b - b^\top Ax + b^\top b \\ &= x^\top (A^\top A)x - 2\, b^\top Ax + b^\top b. \qquad \text{(the two middle terms are equal scalars)}\end{aligned}$$

Take the gradient using our two boxed rules — quadratic term ($A^\top A$ is symmetric) gives $2A^\top Ax$, linear term gives $2A^\top b$, constant gives 0:

$$\nabla_x \|Ax - b\|_2^2 = 2A^\top Ax - 2A^\top b.$$

Set it to zero (bottom of the convex bowl) and solve:

$$A^\top Ax = A^\top b \quad \Longrightarrow \quad \boxed{\hat{x} = (A^\top A)^{-1} A^\top b}.$$

These are the **normal equations**, and $\hat{x}$ is the least-squares solution.

Concept

This is linear regression in one shot. If A is the design matrix X (features plus a ones column) and b is the target vector y, then $\hat{\theta} = (X^\top X)^{-1} X^\top y$ is the best-fit parameter vector — the *same* answer gradient descent crawls toward in Chapter 4, but computed directly. (Gradient descent is still preferred when X is huge, because inverting $X^\top X$ gets expensive.)

How to write it in Python

The normal equation is one NumPy line, and we can *check* the gradient rules numerically (exactly the Lab 2 idea):

```
import numpy as np
# least squares: best-fit line for (1,1),(2,2),(3,3)
A = np.array([[1.,1],[1,2],[1,3]]);  b = np.array([1.,2,3])
theta_hat = np.linalg.inv(A.T @ A) @ A.T @ b       # (A^T A)^{-1} A^T b
print(theta_hat)                                    # -> [0, 1]   (intercept 0, slope 1)

# verify  grad(x^T A x) = 2 A x  with a finite difference
M = np.array([[2.,1],[1,3]])                        # symmetric
f = lambda x: x @ M @ x
x0 = np.array([1., 2.])
h = 1e-5
num = np.array([(f(x0+h*e)-f(x0))/h for e in np.eye(2)])
print(num, 2 * M @ x0)                              # [8 14]  matches 2Ax
```

For heavy problems prefer `np.linalg.lstsq(A, b)` — it solves the normal equations without forming the fragile inverse.

Questions & Variations

Q. What is the gradient of $b^\top x$? Of $x^\top A x$ (symmetric A)?
A. $\nabla(b^\top x) = b$; $\nabla(x^\top A x) = 2Ax$ — the vector versions of $\frac{d}{dx}(ax) = a$ and $\frac{d}{dx}(ax^2) = 2ax$.

Q. Why is the Hessian symmetric?
A. Mixed partials are equal ($\partial^2 f / \partial x_i \partial x_j = \partial^2 f / \partial x_j \partial x_i$), so $H_{ij} = H_{ji}$.

Q. Why is "Hessian = gradient of the gradient" not literally right?
A. The gradient is a vector, and the gradient of a vector is undefined; correctly, you differentiate each component to get one column of H, giving $\nabla^2 f = \nabla(\nabla f)^\top$.

Q. Derive the normal equation.
A. Minimise $\|Ax - b\|^2 = x^\top A^\top A x - 2b^\top A x + b^\top b$; gradient $2A^\top A x - 2A^\top b = 0$ gives $A^\top A x = A^\top b$, so $\hat{x} = (A^\top A)^{-1} A^\top b$.

Q. How does the normal equation relate to linear regression?
A. With $A = X$ (design matrix) and $b = y$, $\hat{\theta} = (X^\top X)^{-1} X^\top y$ is the exact best-fit — the closed-form alternative to gradient descent.

Chapter 7

Batch and Stochastic Gradient Descent

Chapter 4 gave one update rule. But *how much data* do we use to compute the gradient each step? That single choice gives two flavours of gradient descent — the last idea in this course's theory.

7.1 Recap: the batch update uses all the data

Concept

The linear-regression update we derived sums over *every* training example:

$$\theta_j := \theta_j - \alpha \frac{1}{n} \sum_{i=1}^{n} \left(h_\theta(x^{(i)}) - y^{(i)}\right) x_j^{(i)}.$$

That $\sum_{i=1}^{n}$ means: to take *one* step, we look at all n points. That is **Batch** gradient descent.

7.2 Batch gradient descent

In Plain Words

Batch GD uses the *whole* dataset to compute each step. Because it averages over everything, its steps are smooth and point reliably downhill. The price: if you have millions of examples, every single step is expensive — so it is *slow* on big data.

7.3 Stochastic gradient descent

Concept

Stochastic GD (SGD) goes to the opposite extreme: use *one* example at a time. Loop over the data and, for each example i, update immediately:

$$\theta_j := \theta_j + \alpha\big(y^{(i)} - h_\theta(x^{(i)})\big)\, x_j^{(i)}.$$

(This is the single-example version of the same rule — note $(y-h) = -(h-y)$, so the sign just flips the subtraction.) SGD is **faster** (a step costs one example, not n), but because each step trusts a single noisy point, it **may not settle exactly** at the minimum — it jitters around it.

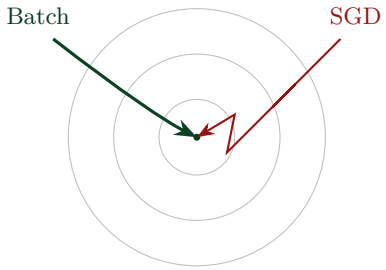


Figure — Both descend to the minimum. Batch takes a smooth, averaged path; SGD zig-zags because each step follows one random example — faster per step, but noisier.

7.4 Batch vs Stochastic: the trade-off

	Batch GD	Stochastic GD
Data per step	all n examples	one example
Step direction	smooth, accurate	noisy
Speed per step	slow (esp. large n)	fast
Convergence	steady to the minimum	jitters near it

In Plain Words

In practice people use a **middle ground** — a small *mini-batch* (say 32 or 64 examples) per step — to get most of Batch's stability with most of SGD's speed. (The course notes formally define only Batch and Stochastic; mini-batch is the practical blend, and it is exactly what Lab 2 implements.)

How to write it in Python

The only code difference is *how much data each update sees*. Same gradient, two loops:

```
import numpy as np
def batch_gd(X, y, alpha=0.02, epochs=400):
    theta = np.zeros(X.shape[1]);  m = len(y)
    for _ in range(epochs):
        grad = (1/m) * (X.T @ (X @ theta - y))      # ALL examples
            per step
        theta = theta - alpha * grad
    return theta

def sgd(X, y, alpha=0.02, epochs=400):
    theta = np.zeros(X.shape[1])
    for _ in range(epochs):
        for i in np.random.permutation(len(y)):     # ONE example per
             step
            xi, yi = X[i:i+1], y[i:i+1]
            grad = xi.T @ (xi @ theta - yi)
            theta = theta - alpha * grad.ravel()
    return theta
```

```
# both recover the true line ~[1, 2] on data y = 2x + 1
```

Batch computes the gradient over the whole dataset before each step; SGD steps after every single example. A mini-batch version just loops over chunks (`X[s:s+bs]`) instead.

Analogy

Deciding how to vote on a new restaurant: **Batch** asks *everyone* in town before each decision — accurate but slow. **SGD** asks *one* random person and acts at once — fast but flighty. **Mini-batch** asks a small focus group — quick and reasonably reliable.

Questions & Variations

Q. What is the difference between Batch and Stochastic gradient descent?
A. Batch uses all n examples to compute each update; Stochastic uses a single example per update.

Q. Give one pro and one con of each.
A. Batch: accurate/steady but slow on large data. SGD: fast per step but noisy and may not converge exactly.

Q. Write the SGD update for linear regression.
A. $\theta_j := \theta_j + \alpha\,(y^{(i)} - h_\theta(x^{(i)}))\,x_j^{(i)}$ — the single-example version of the batch rule.

Q. What is mini-batch gradient descent?
A. A compromise using a small group of examples per update — most of Batch's stability with most of SGD's speed.

Closing: The Whole Story in One Page

You now have the full arc of this course's foundations:

1. **Maths (Ch 0):** functions, slopes, the derivative, vectors, matrices, sums — the alphabet.
2. **What ML is (Ch 1):** learning a function h_θ from examples by minimising a loss $J(\theta)$.
3. **Paradigms (Ch 2):** supervised (regression / classification), unsupervised (clustering / dimensionality reduction), reinforcement.
4. **Linear regression (Ch 3):** the straight-line model and its convex squared-error bowl.
5. **Gradient descent (Ch 4):** roll downhill, $\theta := \theta - \alpha \nabla J$; the cost-gradient derivation; the learning rate.
6. **Gradients & Hessians (Ch 5):** vector calculus, and the least-squares normal equation $\hat{\theta} = (X^\top X)^{-1} X^\top y$.
7. **Batch vs SGD (Ch 6):** how much data per step, and the trade-off.

Every ML method later in the course is a new choice of *model*, *loss*, or *optimiser* — but the loop is always the same: pick a model, measure its loss, and descend the gradient.

Free resources used to build this book

Openly available references that reinforce these chapters (cited, not copied):

Stanford CS229 — Main Notes.
Linear regression, gradient descent, least squares. `https://cs229.stanford.edu/main_notes.pdf`

Mathematics for Machine Learning (Deisenroth et al.).
Vectors, matrices, vector calculus, gradients. `https://mml-book.github.io/book/mml-book.pdf`

Boyd & Vandenberghe, Convex Optimization.
Convexity and why the bowl has one minimum. `https://web.stanford.edu/~boyd/cvxbook/`

S. Ruder, Gradient Descent Optimization Algorithms.
Batch / stochastic / mini-batch and beyond. `https://arxiv.org/abs/1609.04747`

Dive into Deep Learning (d2l.ai).
Runnable chapters on regression and optimisation. `https://d2l.ai/`

End of the Theory Book

Foundations of Machine Learning, built from zero for absolute beginners.
Prepared by Krushna Parmar (202611032), grounded in the course lectures and class notes.